



SHOPWAZIER PROJECT

Interim Progress report

Gavin Higgins
x24159808@student.ncirl.ie

Contents

1.1	Project overview	4
	Background.....	4
	Objective	4
2.1	Project expected deliverables	5
	Application Functionality	6
	Automatic User Access	6
	Barcode Scanning.....	6
	Single Product Price Comparison	6
	Shopping Cart Comparison.....	6
	Price Reporting	6
3.1	Project actual targets met	7
	Research and Planning	7
	REST API Server	7
	Database and connection to Server	7
	API server and Database Health Check	8
	Product Route, Controller and Service Layers	8
	The Product Service Layer.....	8
	The Product Controller and Route Layer.....	8
	Cart Route, Controller and Service Layer	9
	Frontend Design	9
4.1	Deliverables yet to be achieved.....	10
	Cart Route, Controller and Service Layer	10
	Price Route, Controller and Service Layer.....	10
	Automatic User ID system	10
5.1	Main problems encountered.....	11
	Writing Product features vs Cart features	11
	Documentation	11
	Planning Phase.....	11
6.1	Expected Stumbling Blocks	12
	Cart Logic	12
	Presentation	12

7.1 Focus on rubric 13

 Communication 13

 Writing & Presentation of Results 13

 Complexity / Coding Skills..... 13

 Technology..... 13

 Completeness..... 13

 Testing / Evaluation 14

 Project Management..... 14

8.1 Summary of project progression 15

9.1 Signature of author: 15

10.1 References..... 16

1.1 Project overview

Background

The grocery market in Ireland is large and still growing. (irishexaminer, 2025) reported that food prices in Ireland are rising more than twice the general inflation rate, which means that people are spending more money on food and everyday items.

Supermarkets such as Tesco Ireland, Dunnes Stores, Lidl and Aldi Ireland all set their own prices, and these prices can change often. There is no simple app that shows the real in-store prices from all supermarkets in one place.

There is an opportunity to develop ShopWazier to address this gap in the market. ShopWazier will allow users to scan a product barcode and report the price they see in store and in return help people find more accurate price information.

Objective

The objective of this project is to build an app called ShopWazier. The app will allow users to scan a product barcode and see the price of that product in different Irish supermarkets. Users can compare one product across stores to find the cheapest price.

The app will also include a shopping cart feature. Users can add products to their cart and see the total cost of their shopping in different supermarkets. This allows users to compare their full shopping list across stores and see which supermarket is cheaper overall.

Users can report the price they see in store. A price will only update after multiple users confirm the same price. This will help keep the information accurate. (ISPRS, 2018) found that using answers from many different people makes the final result more accurate than using one person. This shows that if several users confirm the same price it is more likely to be correct.

ShopWazier is aimed at everyday shoppers in Ireland who want to save money on groceries. Supermarkets such as Tesco Ireland, Dunnes Stores, Lidl and Aldi all have different prices and these prices can change often. The goal of ShopWazier is to make price comparison simple and help users find the best value for their money.

2.1 Project expected deliverables

At the end of this project several deliverables will be submitted. These include both documents and technical work that show how the ShopWazier system was built and tested.

Table 1:

Document	Description	Date Due	Submitted
Project Proposal	A document that outlines the ShopWazier project idea.	6th February	Yes
Project Requirement Specification	A document that describes what the application will do and how it is expected to perform.	20th February	Yes
Interim Report	A report explaining the current progress of the ShopWazier project.	6h March	Yes
Project Analysis & Design Documentation	A document that explains how the system is structured and how all parts of the application work together.	20th March	No
Live Presentation	A live presentation demonstrating the features and functionality of the completed application.	15th April	No
Video of Final Project	A short video presenting all features of the completed application.	17th April	No
Project Final Report	A document that explains every part of the completed ShopWazier project.	17th April	No
Project Code	The complete source code for the entire application.	17th April	No
Declaration Cover Sheet	A signed declaration confirming that all work submitted in this project is my own original work.	17th April	No

Application Functionality

At the end of this project the ShopWazier app will provide several main features that work together as one complete system.

Automatic User Access

When a user opens the app they will automatically be signed in as a standard user. They will not need to manually register.

Barcode Scanning

Users will be able to scan a product barcode in a supermarket. The app will identify the product and display its reported price.

Single Product Price Comparison

Users will be able to compare the price of a single product across different Irish supermarkets to find the cheapest option.

Shopping Cart Comparison

Users will be able to add multiple products to a shopping cart. The app will calculate the total cost of the basket across different supermarkets so users can see which store is cheapest overall.

Price Reporting

Users can report the price they see in store. A price will only update after multiple users confirm the same value. This helps keep the information accurate.

3.1 Project actual targets met

Research and Planning

At this stage of the project the research and planning phase has been completed. The structure of the ShopWazier application has been analysed and the required tools and technologies have been identified. Initial implementation has also begun. The official (Node.js, 2024) documentation along with a YouTube video by the (Coder, 2025) was used to assist with correctly setting up the backend server and determining which packages were required.

REST API Server

The REST API server has also been completed, knowledge and notes taking previously in the Distributed Systems module along with documentation on Routing from (Express.js, 2024) and using guidance on YouTube videos from (Coder, 2025) helped build the Apps REST API. It also hepled with the guidance to structure the server correctly using routes, controllers and services. Knowledge and notes from Object-Oriented Programming module influenced the decision to use classes in the controller and service layers of the application. The module highlighted the importance of clear structure, reusable code, and building systems that are easy to maintain and add too. Doing it this way allowed for features to be separated into organised files and made it easier to test and implement.

Database and connection to Server

The Database and connection to the server is also complete using mainly notes taking on DDL for SQL in the Database module from college. It helped create the ShopWaziers system database. The database schema and tables were created using MySQL.

Connecting the server to the database was a new challenge during this development. Research was carried out to understand how to establish a database connection using the mysql2 package. The implementation was achieved through the guidance from a You Tube video by (Playbook, 2025) , a report on a website by (Nobi, 2025) and official documentation (MySQL2, 2025)

After completion and testing all documents were also pushed to Github for version control.

API server and Database Health Check

As discussed above the backend server has been successfully created using Express.js. It was decided to set up a health route and controller to confirm that the API is running and that the database connection is working correctly.

Implementing a health check before anything else was a high priority. Although this was not planned in initial planning of the App. The importance of having a health check was found when reviewing documentation in (Express.js, 2024)

The health check for this app was completed using the guidance of an article on a webpage by (Baecker, 2022).

Testing of the Health Route and Controller was also completed. It was done through VS terminal. After completion and testing all documents were pushed to Github for version control.

Product Route, Controller and Service Layers

The Product Service Layer

The Product feature was completed on the Shopwazier App, the code for the service layer was written first because the main part of the feature depends on the database. The App finds products, create them if they did not exist, and check prices. The SQL queries were written and tested directly in MySQL first, to make sure they returned the correct data. These queries were created using the guidance from notes previously taken from the database module in college.

Once the queries were working correctly in SQL, they were added into the service layer. The service contains functions to find a product by barcode, create a new product, and get the price for a store. The controller then calls these functions and sends the final result back as JSON. The formation of the service layer was achieved using guidance of a You Tube video from (Meech-Ward, 2023)

The Product Controller and Route Layer

After the product service layer was completed, the products controller and route layer were built. It was decided that these would be built by re-using as much of the Health layers route and controller code as possible. The product route layer however needed to use the POST route so the user can send data like the barcode and store ID to the server.

Cart Route, Controller and Service Layer

The Carts Route, controller and service layer are only partially built. The cart feature was built one step at a time. Each feature is being written and tested before moving on to the next one. The cart object was created first then the controller and route were added so it could be tested. After that feature of adding items to the cart were developed.

At this stage, the cart can be created, items can be added and removed. However, the cart system is still not fully complete. Code for its other features still need to be written.

Building it step by step makes it easier to test each feature and avoids leaving large parts of the code unfinished.

Frontend Design

The ShopWaziers app screen has also been completed using HTML and CSS. The HTML is used to create the structure of the page, including the main phone container the store selection buttons, the scan and search buttons and the shopping list area where items will appear.

Bootstrap was also used to help with layout and spacing. Bootstrap made it easier to position elements on the page and style buttons.

The CSS file has also been completed. The CSS adds colours, borders spacing and shapes so the layout looks like a simple mobile app screen. It also styles the store buttons with different colours and creates the shopping list box and total price area. Most of this was created using the guidance of notes from the Web Design module.

4.1 Deliverables yet to be achieved

Cart Route, Controller and Service Layer

The cart system still needs to be finished. Some parts of the cart logic still need to be completed. The cart needs to finish getting product information from the database, getting the confirmed store price for that product and calculating the total price of the cart.

Price Route, Controller and Service Layer

The price reporting system also still needs to be completed. The code for the Price Service still needs to be fully written. This part of the App will store user price reports, count how many users reported the same price, update the confirmed price in the database when enough users agree, and remove the reports that were used.

It is planned to use SQL queries similar to how this was done in the Product and Cart service layers. The Price controller and route will then be created to connect the frontend requests to the service layer much the same as how the previous controllers and routes have been implemented.

Each part of this feature will be written and tested separately. This approach is being used because problems were previously encountered when developing the Product service layer. This issue is discussed further in Section [5.1 Main Problems Encountered](#).

Once these parts are completed, the App will then support price reporting, price confirmation, and correct price totals in the cart.

Automatic User ID system

Another deliverable that still needs to be completed is the automatic user ID system. The app will not require users to create an account or sign in. Instead, when a user opens the app for the first time, the App will automatically create a unique user ID for that device. This ID will then be stored on the device and sent with requests to the backend. When a user reports a price, the user ID will be included with the report so the system can track which user submitted it. This will allow the App to count how many different users confirmed the same price.

5.1 Main problems encountered

Writing Product features vs Cart features

When writing code for Product service layer. Most of the code for service layer was written at once and only tested commands in SQL. The service layer was written first, then the controller and route were added. This made it harder and more confusing to find errors because several parts of the code were written at once before being tested and proving to work.

When working on the Cart service layer, a different approach is being used. Each feature is now written one at a time. The features service part of the code is written first, then its controller and route are added, and the feature is tested before moving on to the next one. This has made debugging easier and has saved time and confusion fixing problems.

It also makes it easier to stop when needed after every feature as its like a checkpoint and youll remember easier where you left off. This especially needed to be done as time needs to be spent completing project documentation before its deadline.

Documentation

Before starting the project it was not fully understood how much documentation needed to be completed or how close the deadlines were. A problem currently being faced is that a lot of time is being spent working on documentation which means less time is being spent developing the app. This is an ongoing issue so new approaches such as developing and testing one feature at a time are being used to work more efficiently and not spend hours trying to retrace steps already taking.

Planning Phase

During the planning phase there was an issue in that a lot of time was also spent deciding how the code structure of the app should be organised and what features should be implemented first. At first it seemed like too much time was spent on planning but it is now definitely helping the development. The app is structured so that it is easy to add new features. Also this may be the case later on if time allows. Adding a new feature would mainly involve creating a new route, controller and service layer. Most of the existing code does not need to be touched so there is less of a chance of messing up other features in the app. The planning that was done earlier is now saving time and making it clearer what the next steps are in development of ShopWazier.

6.1 Expected Stumbling Blocks

Cart Logic

An expected stumbling block that has recently been noticed is related to the cart logic and customer price reporting although that has not been started yet. This issue was not identified during the planning stage of the Cart and Prices logic and it is expected to cause minor problems in the near future if it is not addressed.

The cart is currently designed to retrieve product information and prices from the database. But when a customer reports a new price for a product, that reported price should appear in the customer's cart straight away and should not update the price on the database only add a conformation. At the moment the cart will show the confirmed price from the database not the reported customer price. There is no logic nor was it planned to show the price the customer just reported in the cart until been noticed now.

Not much research has been done on this issue yet so that it can be designed out. It is hoped a simple if statement in the cart logic will fix it.

Presentation

Another stumbling block is the final presentation of the project. I have not done many presentations before with only one other presentation ever completed that was on this course.

To improve my confidence and communication skills, I plan to complete a short presentation skills course over the next month to help strengthen my ability to present the project.

7.1 Focus on rubric

Communication

Communication is an area that will require improvement during this project. I have not done many presentations before with only one other presentation completed during this course. Because of this presenting the project clearly may be challenging. I plan to practice the presentation several times and work on improving my presentation skills by attending a course.

Writing & Presentation of Results

The project documentation is being written in a clear and simple way so the development process can be easily understood. The aim is to explain how the system was built and how the different parts work together. Also it is planned to show how each module in college have been combined to make a functional system. Harvard Referencing Style will be used when referencing external sources.

Complexity / Coding Skills

The ShopWazier project includes several parts such as a Node.js backend, Express routes, controllers, service layers and a SQL database. Features like product scanning, price reporting and a shopping cart system are being or have been developed. Building these features and connecting them together requires planning and problem solving.

Technology

The project uses technologies learned during the course such as Node.js, Express, SQL, HTML, CSS and Bootstrap. These technologies work well together for building a web application and allow the system to be structured in a clear and organised way.

Completeness

The project is still in development but the main structure of the system has already been created. Parts of the backend such as the product system and cart system have already been developed and tested. Some features like the price reporting system and remaining cart logic still need to be completed.

Testing / Evaluation

Testing is being done during development rather than waiting until the end. Each feature is written and then tested before moving on to the next one. This makes it easier to find and fix errors early and has already helped reduce debugging time.

Project Management

A lot of time was spent planning how the code should be structured before development started. At first it seemed like too much time was being spent on planning but it is now helping development move faster. The system is structured so new features can easily be added by creating a new route, controller and service layer. One challenge has been balancing development time with the amount of documentation required but the aim is to continue building the project step by step.

In summary I believe the project is moving in the right direction to meet the requirements of the grading rubric. A lot of work and time has already been invested and the aim is to continue developing the system and completing the remaining deliverables.

8.1 Summary of project progression

The aim of this project is to build ShopWazier, a web application that allows users to scan products, report prices and build a shopping cart while shopping in stores. The system allows users to report the price of items they see in shops. When enough users report the same price, the system updates the confirmed price in the database.

Several deliverables are required for this project. These include the project documentation, system design and development of the application. The main features planned for the system include scanning a product barcode, retrieving product information from the database, allowing users to report prices and allowing users to build a shopping cart.

So far, some important parts of the project have been completed. The planning and design stage has been done and the structure of the system has been created. The backend structure using Node.js and Express has also been set up. The product system has been developed which allows products to be retrieved or created when a barcode is inputted. A shopping cart system has also been started which allows a cart to be created and items to be added.

Some parts of the system still need to be completed. This includes finishing the price reporting system and completing the remaining cart logic. Development is continuing one feature at a time so that each part can be tested before moving on.

Some challenges have also been faced during the project. One challenge has been balancing the time between documentation and development. However, the project is continuing to progress and the structure that was planned earlier is helping make development easier.

9.1 Signature of author:

Gavin Higgins

Gavin Higgins

10.1 References

- Algorithms and Advanced Programming (HDSDEV_SEP24_Y2)*. (2025). Retrieved from <https://moodle2025.ncirl.ie/course/view.php?id=53>.
- Baecker, L. (2022). *How to Add a Node.js Health Check Endpoint to Your API Using Express*. Retrieved from hyperping: <https://hyperping.com/blog/how-to-add-a-nodejs-health-check-endpoint-using-express?>
- Coder, T. R. (2025). *2.0 Enterprise Node.js Refactor: Routes, Controllers, Services (Deep Guide)*. Retrieved from You Tube: <https://www.youtube.com/watch?v=cdnYINPre3Y>
- Express.js. (2024). *Express routing guide*. Retrieved from expressjs: <https://expressjs.com/en/guide/routing.html>
- Guide to getResourceAsStream() and FileInputStream in Java*. (2025). Retrieved from <https://www.baeldung.com/java-getresourceasstream-vs-fileinputstream>.
- irishexaminer. (2025). Grocery prices rising more than 6% as key spending season begins. *irishexaminer*. Retrieved from irishexaminer: <https://www.irishexaminer.com/business/economy/arid-41744094.html>
- ISPRS. (2018). *Increasing the Accuracy of Crowdsourced Information on Land Cover via a Voting Procedure Weighted by Information Inferred from the Contributed Data*. Basel: MDPI.
- Meech-Ward, S. (2023). *MySQL Node.js Express*. Retrieved from YouTube: https://www.youtube.com/watch?v=Hej48pi_lOc
- MySQL2. (2025). *MySQL2*. Retrieved from MySQL2: <https://sidorares.github.io/node-mysql2/docs>
- Nobi. (2025). *Getting Started with node-mysql2: A Faster MySQL Client for Node.js*. Retrieved from medium: <https://medium.com/@vimalveeramani/getting-started-with-node-mysql2-a-faster-mysql-client-for-node-js-a4da91be5767>
- Node.js. (2024). *Introduction to Node.js*. Retrieved from Node.js: <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs?utm>
- Object Oriented Software Engineering*. (2024). Retrieved from <https://moodle2025.ncirl.ie/course/view.php?id=626>.
- Playbook, P. B. (2025). *Node.js + MySQL Tutorial for Beginners | Full Setup & First Query*. Retrieved from You Tube: https://www.youtube.com/watch?v=b_e2zNcEtyc
- Reading a CSV File into an Array*. (2025). Retrieved from <https://www.baeldung.com/java-csv-file-array>.